

4.3 Les fonctions en Python

La notion *fonction* en programmation est totalement similaire aux fonctions usuelles en maths. La notion de fonction est très importante en informatique. Les fonctions en général permettent en effet de découper un problème en sous-problèmes, qui peuvent être découpés aussi en petites tâches. Ainsi, les problèmes complexes deviennent plus simples à résoudre.

Exemple de fonction en maths :

$$f(x) : x \rightarrow x^2 + 3x + 2$$

Une fonction prend des paramètres (x pour cet exemple) et renvoie un résultat (la valeur $x^2 + 3x + 2$)

On utilise des fonctions pour :

- Diviser les problèmes complexes en des problèmes faciles à résoudre ;
- Éviter de refaire un traitement plusieurs fois au sein du programme ;
- Rendre le code plus lisible ;
- Permettre la correction des erreurs de façon rapide (On sait dans quelle fonction on a l'erreur)

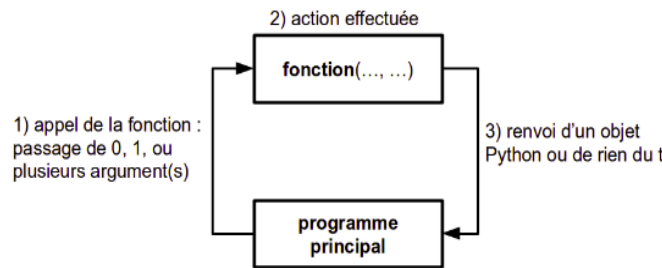
Fonctionnement des fonctions

Nous avons déjà vu quelques fonctions internes de Python :

- `range()`, `input()`, `print()`, `max()`, `min()`

Une fonction en général :

1. À laquelle on passe aucune, une ou plusieurs variable(s) entre parenthèses appelées paramètres ou arguments ;
2. Qui effectue un traitement
3. Qui renvoie un résultat ou rien du tout



Les fonctions peuvent renvoyer ou ne pas renvoyer des résultats à la fin du traitement.

- Les fonctions qui renvoient un résultat sont appelées **fonctions**
- Les fonctions qui ne renvoient pas de résultat sont appelées **Procédures**

Syntaxe des fonctions

 **Structure des fonctions**

```

1 def nomFonction(par_1,..., par_n):
2     instruction_1
3     instruction_2
4     .....
5     instruction_p
6     return resultat
7 r= nomFonction(p_1, p_2, ..., p_n)
8 print(r) # Affichage du résultat
9 #p_1, p_2, ....: sont des variables
  
```

 **Fonction qui calcule le carrée**

```

1 def carre(x):
2     return x**2
3 car_x= carre(7) #affectation à car_x
  
```

 **Structure des procédures**

```

1 def nomProcédure(par_1,..., par_n):
2     instruction_1
3     instruction_2
4     .....
5     instruction_p
6 nomProcédure(p_1,..., p_n) # Appel
  
```

 **Exemple de procédure**

```

1 def soustraire(x,y):
2     p = x-y
3     print("{} - {} = {}".format(x,y,p))
4 soustraire(7,10) # n'a pas de retour
  
```

Les variables x et y sont appelées des arguments positionnels. Ils sont nécessaires lors de l'appel de la fonction. L'ordre doit être respecté.

Lorsque des variables sont définies à l'intérieur du corps d'une fonction, ces variables ne sont accessibles qu'à la fonction elle-même. On dit que ces variables sont des variables locales à la fonction.

Exemple

```
1 def g():
2     y=0
3 g() # appel de la procédure
4 print(y) # undefined variable
```

Les variables définies à l'extérieur d'une fonction sont des variables globales. Leur contenu est visible de l'intérieur d'une fonction, mais la fonction ne peut pas le modifier.

Exemple

```
1 def f():
2     i=7
3 i=0
4 f()
5 print(i) #affiche 0
```

Fonction à arguments optionnels

Il est possible de définir un argument par défaut pour chacun des paramètres. On obtient ainsi une fonction qui peut être appelée avec une partie seulement des arguments attendus.

Exemple

```
1 def exemple(x,y=0):
2     print(x,y)
3 exemple(5,10) # qu'affiche ça ?
4 exemple(10) # qu'affiche ça
```

Fonction anonyme Lambda

Exemple

```
1 f = lambda x : x**2
2 v=f(5)
3 f = lambda x,y: x+y
4 g=f(7,8)
5 print(v,g)
```

Cependant, on peut quand même modifier une variable globale par une fonction à l'aide de l'instruction **global**.

Exemple

```
1 y=0
2 def g():
3     global y
4     y=7
5 g()
6 print(y)
```

Une fonction peut appeler une autre fonction, cette dernière peut appeler une autre fonction et ainsi de suite (et autant de fois qu'on le veut). Une fonction peut même s'appeler elle-même, celle-là s'appelle une fonction récursive.

Exemple

```
1 def fonction1(x):
2     if x%2==0:
3         fonction2(x)
4     else:
5         fonction3(x)
6 def fonction2(x):
7     print(x,"est pair")
8 def fonction3(x):
9     print(x,"est impair")
10 fonction1(15) #appel de la procédure
```

Attention :

Le programme au dessous produira une erreur !
Les arguments optionnels doivent être déclarés à la fin. L'ordre est important !

Exemple

```
1 def exemple2(x,y=0,z):
2     print(x,y,z)
3 exemple2(10,7) # qu'affiche ça ?
```

Le mot-clé **lambda** en Python permet la création de fonctions anonymes (i.e. sans nom et donc non définie par **def**).

4.4 Les assertions

Les assertions sont des tests que l'on effectue pour vérifier si une condition est bien vérifiée ou non (**si elle renvoie True ou False**) avant de poursuivre le traitement de la fonction. Ce test est effectué en utilisant le mot clé **assert**.

Si le condition n'est pas vérifiée, le programme lèvera une **AssertionError**.

Vous pouvez spécifier un message à écrire si le code renvoie *False* comme indiqué dans l'exemple suivant.

Structure 1: utilisation de assert

```
1 #on veut creer une fonction de division n/p
2 def division(n,p):
3     assert p!=0 , "division sur 0 !"
4     return n/p
5 print(division(10,0))
```

4.5 Les fonctions récursives

La méthode la plus efficace pour résoudre un problème complexe est de le dissocier en des sous problèmes faciles à résoudre. La bonne pratique de la programmation est d'utiliser des fonctions pour des problèmes qui se répètent avec des valeurs différentes. Ces fonctions peuvent être itératives (une fonction qui fait un traitement ou qui fait appel à une autre fonction), ou même récursif.

Exemple de récurrence mathématique

Soit la suite récurrente suivante :

$$(U_n) = \begin{cases} 0 & \text{si } n = 0. \\ 3U_{n-1} + 1 & \text{sinon.} \end{cases}$$

Pour calculer donc le n^{eme} terme de U_n on calcule les termes $U_{n-1}, U_{n-2}, \dots$:

$$U_n = 3(U_{n-1}) + 1 \quad (2.1)$$

$$= 3(3(U_{n-2}) + 1) + 1 \quad (2.2)$$

$$= 3(3(3(U_{n-3}) + 1)) + 1 + 1 \quad (2.3)$$

$$= \quad (2.4)$$

$$= 3(3(3(.....(3U_0) + 1)) + 1) + 1 \quad (2.5)$$

Définition 1

- Une fonction récursive est une fonction qui s'appelle elle-même soit de façon directe ou indirecte.
- Une fonction récursive doit contenir un cas où les appels récursifs s'arrêtent (**cas de base**)

Structure des fonctions récursives

```
1 def Fonction(par1, par2, ..., parn):
2     if [condition]:
3         return valeur # cas de base
4     else:
5         instruction 1
6         instruction 2
7         .....
8         return Fonction(x1,x2,...,xn) # appel récursif
```

Exemple de la fonction factorielle

```
1 def factorielle(n):
2     if n==0: # cas de base
3         return 1
4     else:
5         return n*factorielle(n-1) # appel récursif
```

4.6 Types de récursivités

Récursivité simple

Pour ce type de récursivité on fait un seul appel récursif pour la fonction P dans le corps d’une fonction récursive P ;

 **Fonction de puissance**

$$x^n = \begin{cases} 1 & \text{si } n = 0 \\ x * x^{n-1} & \text{sinon.} \end{cases}$$

Récursivité multiple

Pour ce type de récursivité on fait un plusieurs appels récursifs pour la fonction P dans le corps d’une fonction récursive P ;

 **Exemple**

$$U_n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ U_{n-1} + U_{n-2} & \text{sinon.} \end{cases}$$

Pour ce genre de fonctions, on dessine l’arbre de récurrence de U_n pour voir les différents appels faits par la fonction initiale. Exemple pour $n = 4$:

 **Exercices**

- Programmer en Python les fonctions itératives suivantes $factorielle(n)$, $Puissance(x, n)$
- Programmer en Python les fonctions récursives suivantes $factorielle(n)$, $Puissance(x, n)$